

POLITECNICO
MILANO 1863

Ingegneria del Software

Ingegneria Informatica – Prof. San Pietro

Esercitazione:

Ereditarietà & Interfacce

My Contacts:
Giuseppe Sorrentino
giuseppe.sorrentino@polimi.it

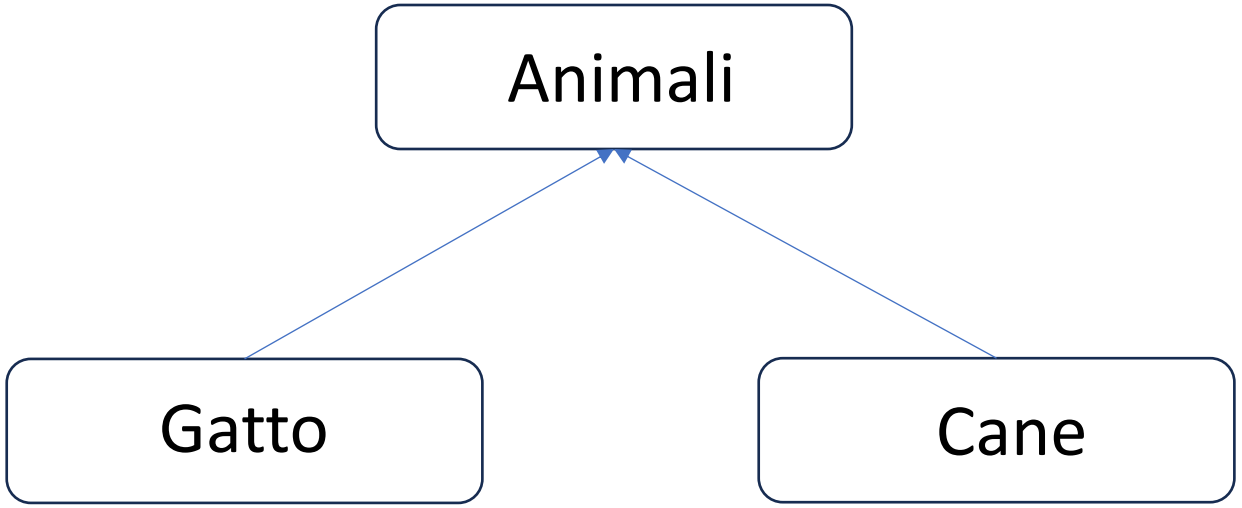
Example: Geometric Shapes

- Si implementi una gerarchia di classe che rappresenti delle forme geometriche.
- In particolare si implementi il calcolo dell'area, del perimetro e la rappresentazione su schermo.

Cosa dovrei usare?

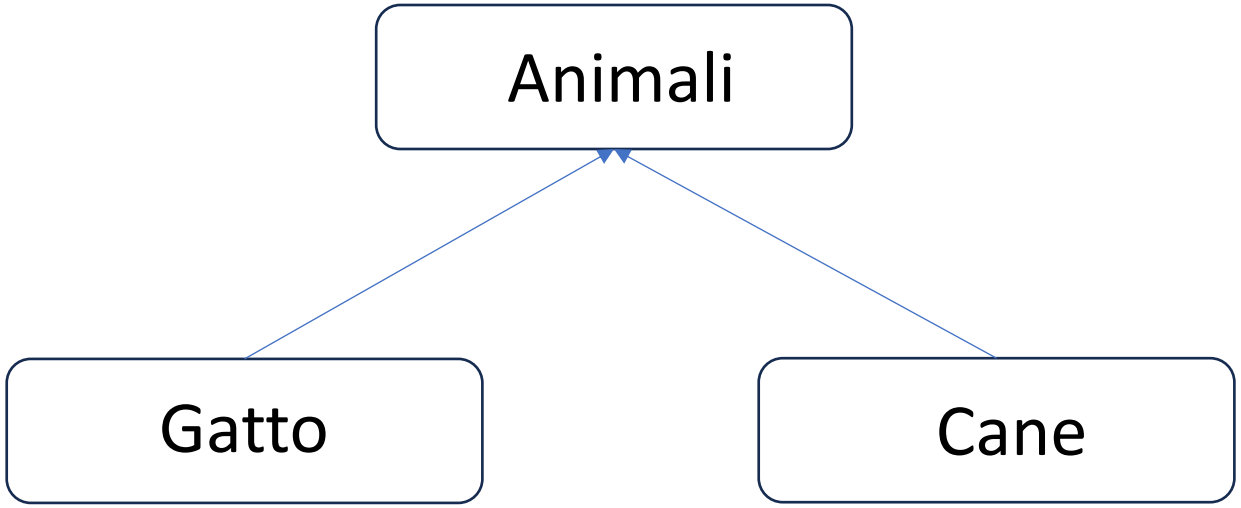
Ereditarietà ed Interfacce

- Consideriamo questa gerarchia



Ereditarietà ed Interfacce

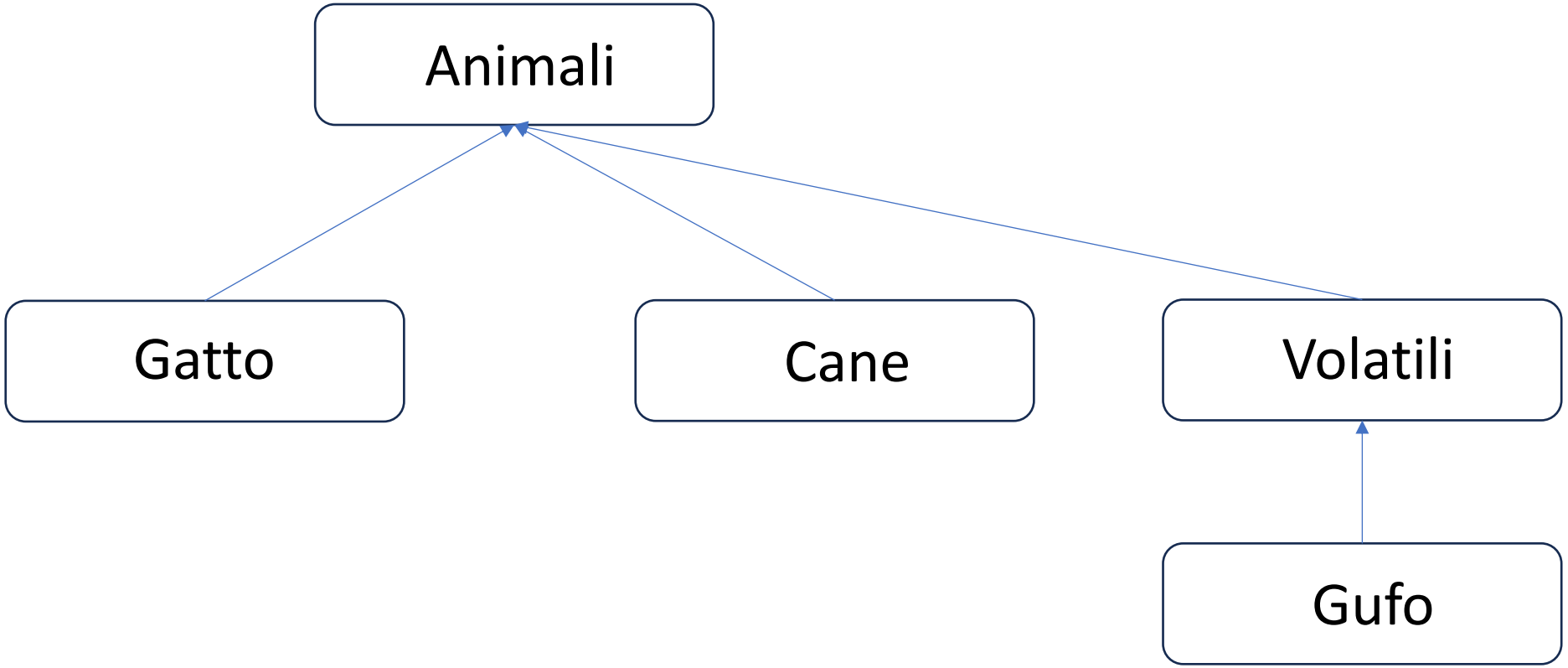
- Consideriamo questa gerarchia



- Come possiamo estendere questa gerarchia per includere volatili?

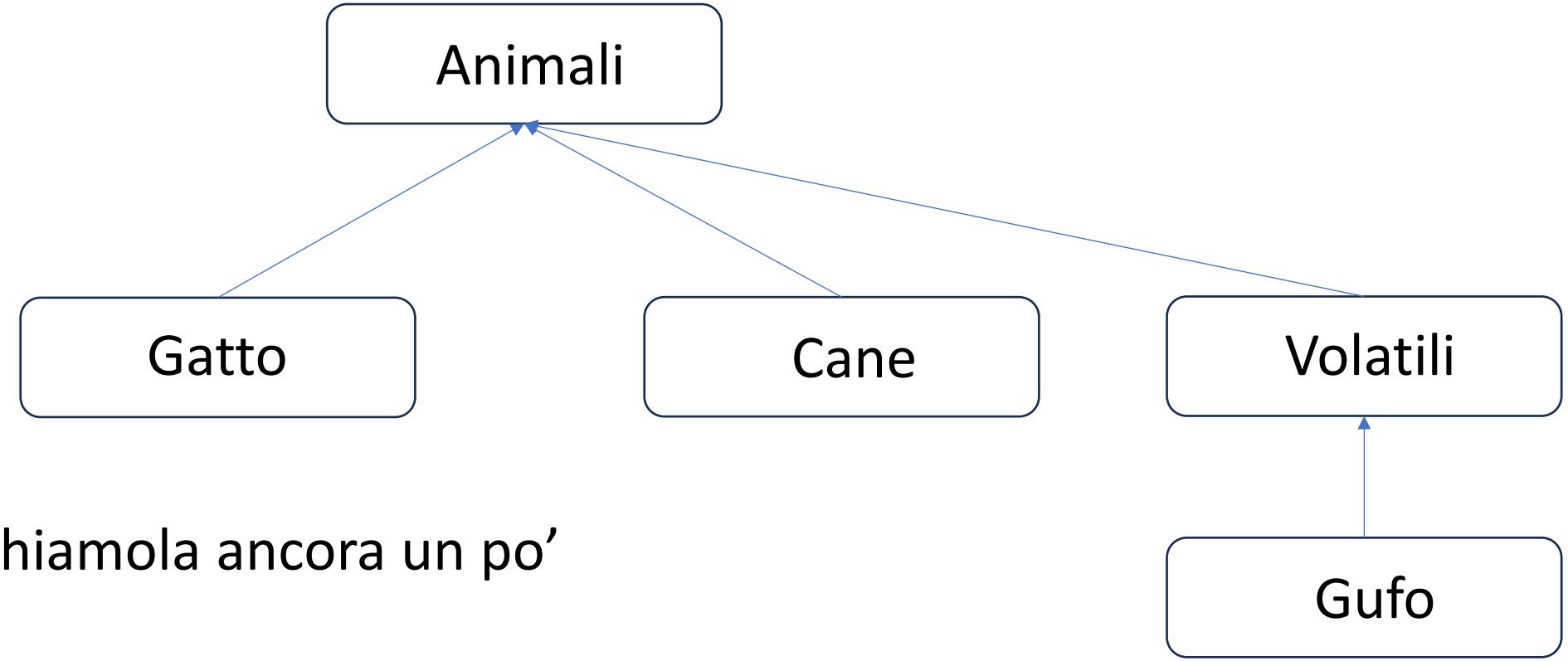
Ereditarietà ed Interfacce

- Consideriamo questa gerarchia



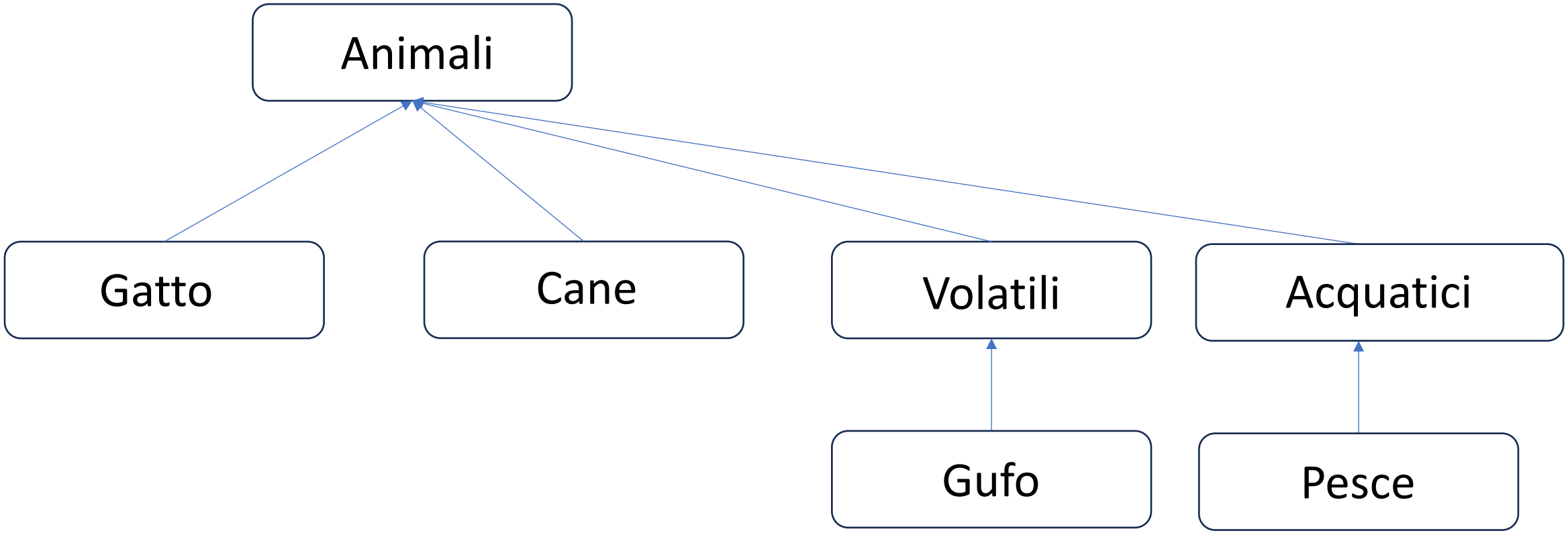
Ereditarietà ed Interfacce

- Consideriamo questa gerarchia

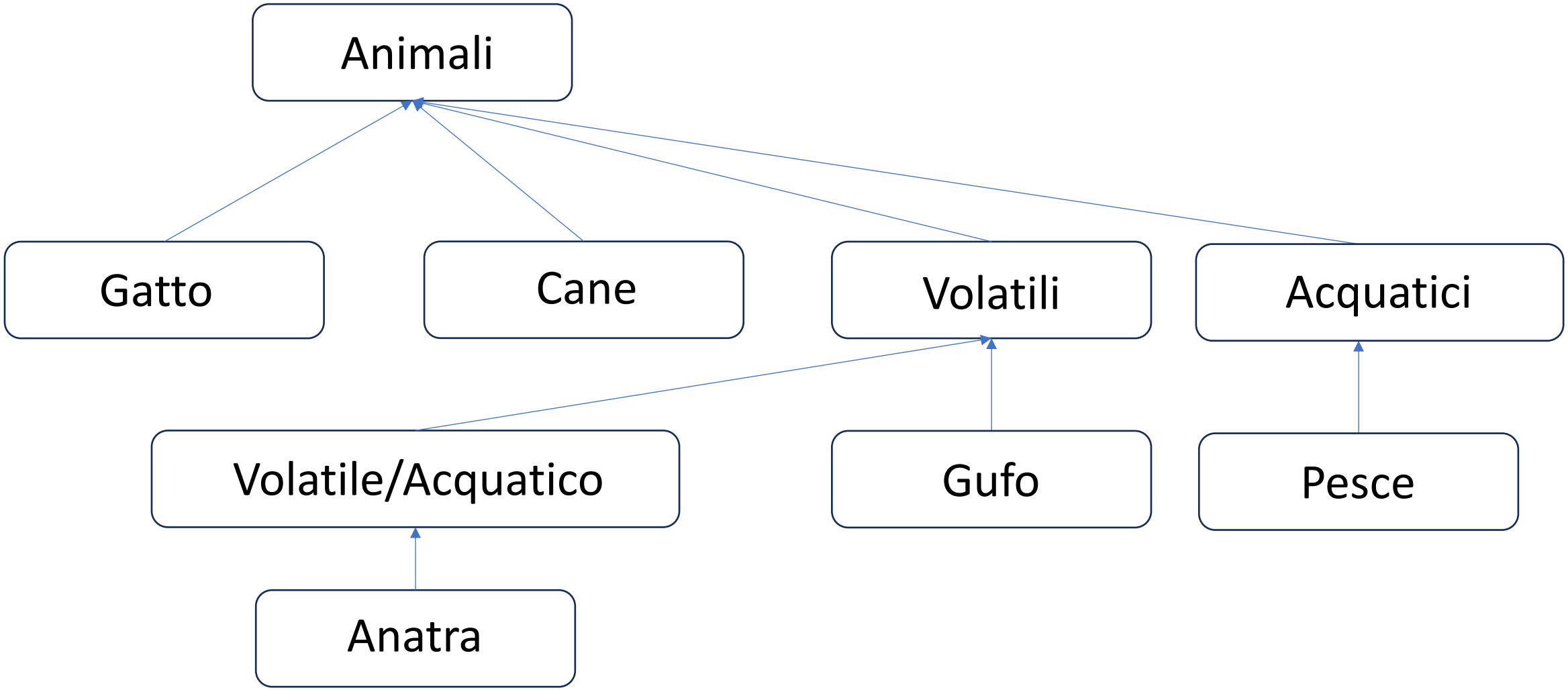


- Ora complichiamola ancora un po’

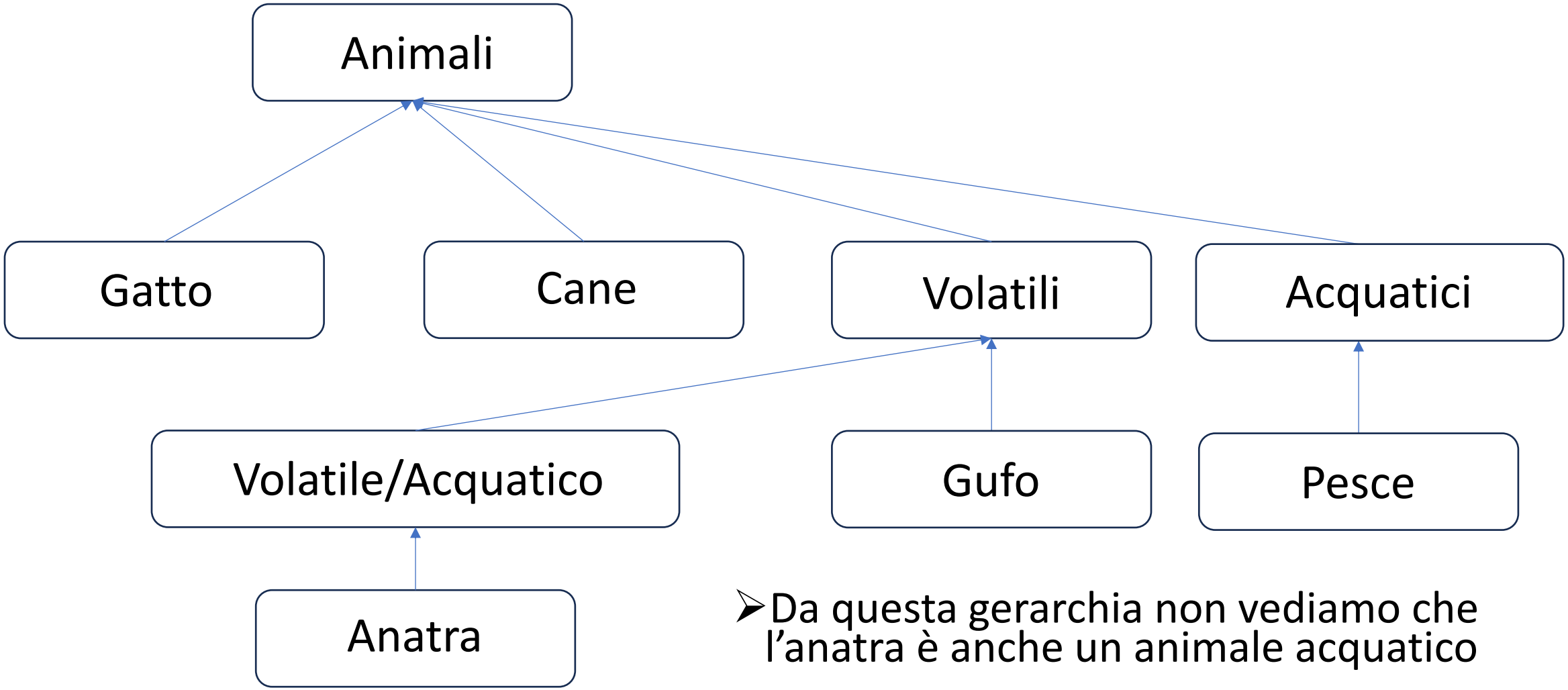
Ereditarietà ed Interfacce



Ereditarietà ed Interfacce



Ereditarietà ed Interfacce



- Da questa gerarchia non vediamo che l'anatra è anche un animale acquatico

Soluzione: Ereditarietà Multipla

- Non è direttamente supportata in Java
  (e.g. potrebbero esserci problemi come il «diamond problem»)
- Può essere realizzata attraverso le **interfacce**

Implementazione

```
1 public class Anatra extends Animale implements Volatile, Acquatico {
2 // Deve implementare i metodi definiti in Volatile ed Acquatico
3 @Override
4 public void vola(){ ... }
5 @Override
6 public void nuota(){ ... }
7 }
```

Classi vs Interfacce

- Una classe rappresenta il fatto che un oggetto è qualcosa.
- Una interfaccia rappresenta il fatto che la classe implementa un certo comportamento.

Interfacce come tipi

- Anche le interfacce possono essere usate come tipo di un riferimento.
- Come tipo statico, un'interfaccia espone tutti i metodi che definisce.
- L'implementazione utilizzata dipende dal tipo dinamico della classe (Binding dinamico).
- Per gli assegnamenti dobbiamo considerare le relazioni di tipo/sotto-tipo.

Attenzione!

- NON CONFONDERE classi astratte ed interfacce

Attenzione!

- NON CONFONDERE classi astratte ed interfacce

1. Una classe può estendere una sola classe astratta, ma può implementare più interfacce.

Attenzione!

- NON CONFONDERE classi astratte ed interfacce

1. Una classe può estendere una sola classe astratta, ma può implementare più interfacce.
2. Una classe astratta può contenere codice e metodi implementati, un’interfaccia NON deve contenere implementazioni.

Attenzione!

- NON CONFONDERE classi astratte ed interfacce

1. Una classe può estendere una sola classe astratta, ma può implementare più interfacce.
2. Una classe astratta può contenere codice e metodi implementati, un’interfaccia NON deve contenere implementazioni.
3. Una variabile dichiarata all’interno di un’interfaccia è di fatto una costante.

Attenzione!

- NON CONFONDERE classi astratte ed interfacce

1. Una classe può estendere una sola classe astratta, ma può implementare più interfacce.
2. Una classe astratta può contenere codice e metodi implementati, un'interfaccia NON deve contenere implementazioni.
3. Una variabile dichiarata all'interno di un'interfaccia è di fatto una costante.
4. I metodi in un'interfaccia sono **sempre** public.

Keyword

implements -> una classe *implementa* un'interfaccia

extends -> una classe *estende* al più una classe.
un'interfaccia può *estendere* una o più interfacce.

Implementazione

```
1 public abstract class Animale {
2 }
```

Implementazione

```
1 public class Gatto extends Animale {
2 }
```

Implementazione

```
1 public class Anatra extends Animale implements Volatile, Acquatico {
2 // Deve implementare i metodi definiti in Volatile ed Acquatico
3 @Override
4 public void vola(){ ... }
5 @Override
6 public void nuota(){ ... }
7 }
```

Quali operazioni sono consentite?

Svolto in Aula

```
1 Animale g = new Gatto();
2 Pesce p = new Pesce();
3 Anatra a = new Anatra();
4 Volatile g1 = g;
5 Volatile v = a;
6 Nuotatore n = a;
7 g.vola();
8 g.nuota();
9 v.vola();
10 n.nuota();
```

Quali operazioni sono consentite?

Svolto in Aula

```
1 Animale g = new Gatto();
2 Pesce p = new Pesce();
3 Anatra a = new Anatra();
4 Volatile g1 = g; //No, g non e' istanza di implementazione di Volatile
5 Volatile v = a;
6 Nuotatore n = a;
7 g.vola(); //No, g non vola
8 g.nuota(); //No, g non nuota
9 v.vola(); //OK
10 n.nuota(); //OK
```

Modificatori di Visibilità

	Classe	Package	Sottoclasse (Stesso package)	Sottoclasse (Package diversi)	Everywhere
Public	✓	✓	✓	✓	✓
Protected					
Package					
Private					

Modificatori di Visibilità

	Classe	Package	Sottoclasse (Stesso package)	Sottoclasse (Package diversi)	Everywhere
Public	✓	✓	✓	✓	✓
Protected	✓	✓	✓	✓	✗
Package					
Private					

Modificatori di Visibilità

	Classe	Package	Sottoclasse (Stesso package)	Sottoclasse (Package diversi)	Everywhere
Public	✓	✓	✓	✓	✓
Protected	✓	✓	✓	✓	✗
Package	✓	✓	✓	✗	✗
Private					

Modificatori di Visibilità

	Classe	Package	Sottoclasse (Stesso package)	Sottoclasse (Package diversi)	Everywhere
Public	✓	✓	✓	✓	✓
Protected	✓	✓	✓	✓	✗
Package	✓	✓	✓	✗	✗
Private	✓	✗	✗	✗	✗

Esercizio Visibilità 1: Modificatori di visibilità:

Svolto in Aula

```
1 package a;
23
..... class First {
4 ..... int x;
5 ..... int y;
67
..... void h() {
8 y = −1;
9 }
10 }
11
12 ..... class Second extends First {
13 ..... void f(int x) {
14 this.x = x;
15 h();
16 }
17 }
```

```
1 package b;
2 import a.∗;
34
class Third {
5 public static void main(String[] s)
{
6 Second z = new Second();
7 z.f(3);
8 }
9
10 class Fourth extends First {
11 void g() {
12 h();
13 }
14 }
```

Esercizio Visibilità 1: Modificatori di visibilità:

Svolto in Aula

```
1 package a;
2
3 public class First {
4 int x;
5 private int y;
67
protected void h() {
8 y = −1;
9 }
10 }
11 public class Second extends First {
12 public void f(int x) {
13 this.x = x;
14 h();
15 }
16 }
```

```
1 package b;
2 import a.∗;
34
class Third {
5 public static void main(String[] s)
{
6 Second z = new Second();
7 z.f(3);
8 }
9
10 class Fourth extends First {
11 void g() {
12 h();
13 }
14 }
```

Modificatori di visibilità

```
1 //Primo file
2 package a;
34
public class C1 {
56
public void m1() { }
7 protected void m2 () { }
8 private void m3 () { }
9 }
```

main

```
1 C2 x = new C2(); //Dire se corretta e
2 x.m1(); //che output produrrebbe
```

Svolto in Aula

```
1 //Secondo file
2 package b;
3 import a.*;
45public class C2 extends C1 {
67
public void m1() {
8 System.out.print("Salve");
9 m2();
10 m3();
11 }
12
13 protected void m2 () {
14 System.out.print(", mondo");
15 }
16
17 private void m3() {
18 System.out.print("!");
19 }
```

Modificatori di visibilità

Svolto in Aula

- La definizione di C2 e' corretta.
- Il metodo m1 puo' utilizzare i metodi m2 e m3.
- L'output e' "Salve, mondo!".

Modificatori di visibilità

```
1 //Primo file
2 package a;
3
4
public class C1 {
5
6
public void m1() { }
7 protected void m2 () { }
8 private void m3 () { }
9 }
```

```
1 C2 x = new C2(); //Dire se corretta e
2 x.m1(); //che output produrrebbe
```

```
1 package b;
2 import a.*;
3
4 public class C2 extends C1 {
5
6 public void m1() {
7 System.out.print("Salve");
8 m2();
9 m3();
10 }
11
12 protected void m2() {
13 System.out.print(", mondo");
14 }
15
16 //Nessuna definizione di m3
17 }
```

Svolto in Aula

Modificatori di visibilità

Svolto in Aula

- In questo caso C2 non ha visibilità di m3
- Errore di compilazione!!

Modificatori di visibilità

```
1 //Primo file
2 package a;
3
4
public class C1 {
5
6
public void m1() { }
7 protected void m2 () { }
8 private void m3 () { }
9 }
```

```
1 C2 x = new C2(); //Dire se corretta e
2 x.m1(); //che output produrrebbe
```

Svolto in Aula

```
1 package b;
2 import a.*;
3
4public class C2 extends C1 {
5
6
public void m1() {
7 System.out.print("Salve");
8 m2();
9 m3();
10 }
11
12 //Nessuna definizione di m2
13
14 private void m3() {
15 System.out.print("!");
16 }
17 }
```

Modificatori di visibilità

Svolto in Aula

- In questo caso C2 ha visibilità di m2 definito in C1
- L'output sarà *Salve*!

Modificatori di visibilità

```
1 //Primo file
2 package a;
34
public class C1 {
56
public void m1() { }
7 protected void m2 () { }
8 private void m3 () { }
9 }
```

main

```
1 C2 x = new C2(); //Dire se corretta e
2 x.m1(); //che output produrrebbe
```

Svolto in Aula

```
1 //Secondo file
2 package b;
3 import a.*;
45public class C2 extends C1 {
67
public void m1() {
8 System.out.print("Salve");
9 m2();
10 m3();
11 }
12
13 protected void m2 () {
14 System.out.print(", mondo");
15 }
16
17 private void m3() {
18 System.out.print("!");
19 }
```

Esercizio Visibilità 2: Modificatori di visibilità

Svolto in Aula

```
1 package c;
2 import a.*;
3 import b.*;
4 public class C3 {
5 public static void main(String[] s){
6 C1 c1;
7 C2 c2;
8 Object o;
9 c1 = new C1();
10 c1.m1();
11 c2 = new C2();
12 c2.m2();
13 c1 = c2;
14 c1.m1();
15 c2 = new C1();
16 o = new C1();
17 c2 = (C2)o;
18 o = new C2();
19 c1 = (C1) o;
20 c1.m1();
21 }
22 }
```

1. Indicare gli errori a compile-time ed eliminare le istruzioni che li generano
2. Dire se ci sono errori a run-time ed eliminare le istruzioni che li generano
3. Determinare l'output del programma

Esercizio Visibilità 2: Modificatori di visibilità

Svolto in Aula

```
1 package c;
2 import a.*;
3 import b.*;
4 public class C3 {
5 public static void main(String[] s){
6 C1 c1;
7 C2 c2;
8 Object o;
9 c1 = new C1();
10 c1.m1();
11 c2 = new C2();
12 c2.m2(); //m2 protected, errore a tempo di compilazione ( m2 non è visto da c2)
13 c1 = c2;
14 c1.m1();
15 c2 = new C1(); //C1 superclasse di C2, errore a tempo di compilazione
16 o = new C1();
17 c2 = (C2)o; //Errore a run−time: o istanza di C1
18 o = new C2();
19 c1 = (C1) o;
20 c1.m1();
21 }
22 }
```

Esercizio Visibilità 3

Svolto in Aula

Quale di queste operazioni è sbagliata?

```
1 class Persona{
2 String saluto(){
3 return "Buongiorno";
4 }}
5 class PersonaEducata extends Persona
{
6 String saluto() {
7 return "Buongiorno a lei";
8 }}
9 class PersonaMaleducata extends Persona {
10 String saluto() {
11 return "Faccia silenzio!";
12 }}
13 class PersonaMaleducatissima extends PersonaMaleducata
{
14 String saluto() {
15 return "Non mi rompa!";
16 }}
```

```
1 Persona p = new Persona();
2 PersonaEducata pe =new
PersonaEducata();
3 PersonaMaleducata pm =new
PersonaMaleducata();
4 PersonaMaleducatissima pmm=new
PersonaMaleducatissima();
5 p.saluto();
6 pe = p;
7 p = pe;
8 p.saluto();
9 pe = pm;
10 pe.saluto();
11 pm.saluto();
12 p = new PersonaMaleducata();
13 p.saluto();
14 pm = p;
15 pmm = (PersonaMaleducatissima) pm;
16 pmm.saluto();
```

Esercizio Visibilità 3

Svolto in Aula

```
1 class Persona{
2 String saluto(){
3 return "Buongiorno";
4 }}
5 class PersonaEducata extends Persona
{
6 String saluto() {
7 return "Buongiorno a lei";
8 }}
9 class PersonaMaleducata extends Persona {
10 String saluto() {
11 return "Faccia silenzio!";
12 }}
13 class PersonaMaleducatissima extends PersonaMaleducata
{
14 String saluto() {
15 return "Non mi rompa!";
16 }}
```

Quale di queste operazioni è sbagliata?

```
1 Persona p = new Persona();
2 PersonaEducata pe =new
PersonaEducata();
3 PersonaMaleducata pm =new
PersonaMaleducata();
4 PersonaMaleducatissima pmm=new
PersonaMaleducatissima();
5 p.saluto();
6 pe = p;
7 p = pe;
8 p.saluto();
9 pe = pm;
10 pe.saluto();
11 pm.saluto();
12 p = new PersonaMaleducata();
13 p.saluto();
14 pm = p;
15 pmm = (PersonaMaleducatissima) pm;
16 pmm.saluto();
```

Interfaccia Iterator <E>

- L'interfaccia Iterator <E> permette di scandire e rimuovere oggetti da una collezione.
- E' composta dai metodi:

```
1 boolean hasNext()
2 E next();
3 void remove();
```

- Esempio

```
1 Iterator<String> iterator = list.iterator();
2 while(iteratore.hasNext()){
3 string = iterator.next();//Elemento successivo
4 iterator.remove();//Rimuovi elemento corrente
5 }
```

Interfaccia Iterable <T>

- L’interfaccia Iterable<T> permette di implementare il for generalizzato (for each).
- E’ composta dal metodo:

Iterator<T> iterator();

- E’ possibile usare il for generalizzato (for each) su oggetti che implementano tale interfaccia.

Esercizio Base

Svolto in Aula

Definisci una interfaccia 'operazione' e le sue classi derivate(sub. div, mul) in modo che da ciascuna sia possibile eseguire la corrispondente operazione aritmetica (di addizione, divisione, moltiplicazione) istanziando un oggetto di classe 'operazione' e due operandi. Realizzare anche, una classe astratta che implementi le funzionalità comuni come la stampa a video del risultato.

http://www.edutecnica.it/informatica/abstractx/5.htm

Esercizio GeometricShape

Implementare una interfaccia TwoDimensionalCanvas che dichiari i metodi implementati dalla classe BasicCanvas, fornita nel materiale didattico. Inoltre, definire una classe astratta Shape, e due sue estensioni, Square e Circle, data la classe Point, definita nel materiale didattico.

http://www.edutecnica.it/informatica/abstractx/5.htm

Esercizio 4: People & Students

- Implementare una classe per rappresentare Persone e Studenti
- Tener traccia del nome e della data di nascita della persona.
- Gli student sono persone che hanno una lista di esami “tentati”.

Esercizio 4: People & Students

- Implementare una classe per rappresentare Persone e Studenti
- Tener traccia del nome e della data di nascita della persona.
- Gli student sono persone che hanno una lista di esami “tentati”.

HINT: Clone()

clone() è un (protected) method definito in Object per restituire una copia dell’oggetto.

- Per usare questo metodo, la classe deve implementare l’interfaccia Clonable, altrimenti l’implementazione di default lancia soltanto un’eccezione.

- Può essere sostituito da un “copy constructor”.

Esercizio 5: Complex Number

- Fare una classe per gestire i numeri complessi(class Complex).
- Che funzionalità dovrebbe offrire?

Esercizio 5: Complex Number

- **Operations**:
  - Sum, difference
  - Equality (given the requested precision)
  - Modulo, phase
  - Complex conjugate
  - n-th radix

Esercizio 5: Complex Number

- **Representation**:

  - Parte reale e parte immaginaria

Esercizio 5: Complex Number

- **Representation**:

  - Parte reale e parte immaginaria

- **Attributes**:

  - re, im (double); Rappresentano parte reale ed immaginaria, rispettivamente

Esercizio 5: Complex Number

- **Representation**:

  - Parte reale e parte immaginaria

- **Attributes**:

  - re, im (double); Rappresentano parte reale ed immaginaria, rispettivamente

- **Methods**

  - Constructors: inizializza l'oggetto.

  - Observers: restituisce l'informazione sullo stato dell'oggetto.

  - Producers: restituisce nuovi oggetti.

Esercizio 5: Complex Number

- **Representation**:

  - Parte reale e parte immaginaria

- **Attributes**:

  - re, im (double); Rappresentano parte reale ed immaginaria, rispettivamente

- **Methods**

  - Constructors: inizializza l'oggetto.

  - Observers: restituisce l'informazione sullo stato dell'oggetto.

  - Producers: restituisce nuovi oggetti.

- The class should be ***Immutable***

  - Non deve essere possible modificare lo stato dell'oggetto **dopo** l'instanziazione.

  - Nessun metodo deve poter modificare lo stato dell'oggetto dopo l'instanziazione.

Class Complex

```
1 public class Complex {
2         private final double re, im;
3         public Complex() { ... }
4         public Complex(double re, double im) { ... }
5         public double re() { ... }
6         public static Complex fromPolarForm(double abs, double phase) { ... }
7         public double im() { ... }
8         public double abs() { ... }
9         public double phase() { ... }
10        public Complex conjugate() { ... }
11        public Complex sum(Complex other) { ... }
12        public Complex diff(Complex other) { ... }
13        public Complex mult(Complex other) { ... }
14         public boolean almostEquals(Complex other, double precision) { ... }
15         public ComplexSet getRoots(int degree) { ... }
16        }
```

Torniamo alle forme geometriche

- Si modifichi l'implementazione delle forme geometriche per permettere di "cambiare aspetto"
- Si definiscano opportunamente le interfacce ***Rotatable*** e ***Resizable***

Interfacce Rotatable e Resizable

```
1 public interface Rotatable {
2 Shape rotate(double angle);
3 }
4
5 public interface Resizable {
6 Shape resize(double scale);
7 }
```

- Una forma modificata è ancora una forma
- Il tipo di ritorno è un generico *Shape*